

README

HelixCode Developer Guide — Zero-Bluff Phase 5

Audience: Contributors and integrators working on the HelixCode codebase. **Companion:** Architecture deep-dive at [docs/improvements/](#) and [helix_code/docs/architecture/](#). **Mandate:** Every contribution MUST comply with the Constitution (CONSTITUTION.md) and the Zero-Bluff anchors (Article XI §11.9, CONST-035, CONST-046). **Last updated:** 2026-05-12

1. Repository Layout

HelixCode is a **meta-repository** of governance plus submodules, with the actual Go application in `helix_code/helix_code/`.

<code>helix_code/</code>	← meta-repo (governance + submodule)
├── <code>CLAUDE.md / AGENTS.md / CONSTITUTION.md</code>	
├── <code>helix_code/</code>	← inner Go module (the application)
│ ├── <code>cmd/</code>	← CLI + server entry points
│ ├── <code>internal/</code>	← ~55 internal packages (the domain)
│ ├── <code>applications/</code>	← desktop / mobile / TUI
│ └── <code>tests/</code>	← unit, integration, e2e, security,
├── <code>helix_qa/</code>	← QA submodule
├── <code>challenges/</code>	← Challenge bank
├── <code>containers/</code>	← Docker/container artefacts
└── <code>docs/</code>	← meta-level docs (this directory)

Cardinal rule: paths in instructions almost always refer to the inner module — `internal/`
`auth` means `helix_code/internal/auth`.

2. Adding a Feature — The F01–F30 Pattern

Every shipped feature followed this lifecycle. Use the same structure for new work.

1. **Brainstorm** — capture user intent in `docs/superpowers/specs/<date>-<id>-design.md` using the `superpowers:brainstorming` skill.
2. **Spec** — formalise scope, components, success criteria, and out-of-scope items.
3. **Plan** — break into tasks at `docs/superpowers/plans/<date>-<id>-plan.md`. Each task has TDD-shaped acceptance criteria.
4. **TDD** — `superpowers:test-driven-development:red` → `green` → `refactor`, mocks only in unit tests.
5. **Challenge** — every feature ships a Challenge in `challenges/banks/<feature>/` that exercises the real workflow.
6. **Evidence** — append runtime evidence to `docs/improvements/PROGRESS.md`.
7. **Commit + push** to all four remotes (origin / github / gitlab / upstream) per CONST-043.

The 30-feature programme is documented in docs/superpowers/specs/2026-05-04-cli-agent-fusion-synthesis-design.md.

3. The Tool Interface (F05, F21)

Tools are pluggable units the agent can invoke. Every tool implements:

```
// helix_code/internal/tools/types.go
type Tool interface {
    Name() string
    Description() string
    Schema() *jsonschema.Schema
    RequiresApproval() approval.ApprovalLevel // read-only|
        edit|run|all
    Validate(args map[string]any) error
    Execute(ctx context.Context, args map[string]any) (*Result,
        error)
}
```

RequiresApproval() defines the gate level the approval matrix evaluates. New tools default to LevelEdit unless they only read.

Register a tool:

```
// helix_code/internal/tools/<your_tool>.go
func init() {
    tools.MustRegister(&MyTool{...})
}
```

The registry runs through internal/tools/registry.go and is consumed by the executor in internal/agent/.

4. Slash Commands (F09)

Commands live in internal/commands/. Each command implements:

```
type Command interface {
    Name() string // "/"approval"
    Description() string
    Subcommands() []Subcommand // "/"approval status", "/"
        approval set <mode>"
    Execute(ctx context.Context, args []string) error
}
```

User-defined commands are loaded from ~/.config/helixcode/commands/*.md at startup; see internal/commands/loader.go.

5. Skill System (F10)

Skills are Markdown documents with YAML frontmatter loaded by `internal/skills/`:

```
---
name: my-skill
description: When user asks for X, do Y
---
# Skill body
```

Skill matching is performed by the LLM against frontmatter `description`. Skills are not auto-invoked — the agent must call the Skill tool explicitly.

6. Subagent Dispatch (F15)

`internal/task/manager.go` orchestrates parallel subagents. The Task tool, exposed to the agent, accepts a task brief and returns the subagent's final message.

Key types: `TaskManager`, `Task`, `Checkpoint`, `TaskType`. Persistence is via PostgreSQL with Redis caching (`internal/task/manager_db.go`).

Use cases:

- Parallel independent edits (`superpowers:dispatching-parallel-agents`).
 - Long-running analyses that should not block the REPL.
 - Pre-flight checks that may be retried.
-

7. Sandboxed Execution (F14, F21)

`internal/sandbox/` (lives inside `internal/agent/` after refactor — check `internal/approval/manager.go` for the integration point) provides three profiles:

Profile	Filesystem	Network
read-only	RO mount of workspace	None
workspace-write	RW for workspace, RO elsewhere	None
danger-full-access	RW everywhere	Unrestricted

`full-auto approval` mode REFUSES to start unless a non-`danger-full-access` profile is active. This invariant is enforced in `approval.ApprovalManager` (`ErrSandboxRequired`).

8. Telemetry & Observability (F16)

`internal/telemetry/` wraps OpenTelemetry SDK. Spans wrap every tool invocation; metrics expose tokens-in, tokens-out, request latency, sandbox denials, and approval-gate denials.

Enable:

```
HELIXCODE_OTEL_ENDPOINT=http://localhost:4317 ./bin/cli
```

For dashboards see [docs/deployment_guide/README.md](#) §4.

9. Anti-Bluff Discipline (CONST-035, CONST-046)

When writing tests:

- **No mocks in integration/E2E tests.** Use real PostgreSQL, real Redis, real Ollama via `make test-infra-up`.
- **No metadata-only assertions.** A “PASS” must come from positive runtime evidence — assert on actual output, not on `err == nil`.
- **No hardcoded English strings.** All user-visible text must be LLM-generated, i18n-loaded, or composed from verifier metadata (CONST-046).
- **No simulation.** No `for now`, `simulated`, `TODO implement`, `placeholder` strings in `internal/` or `cmd/`.

Smoke check (must always pass):

```
grep -rn "simulated\|for now\|TODO implement\|placeholder" \
  helix_code/internal helix_code/cmd && echo "BLUFF FOUND" ||
  echo "clean"
```

Phase 4 added mutation-tested content assertions; see `challenges/banks/zero-bluff/` for templates.

10. Governance Documents

Every owned-by-us submodule MUST carry:

- `CONSTITUTION.md` with anti-bluff anchor (Article XI §11.9, CONST-035, CONST-045, CONST-046).
- `CLAUDE.md` with cascaded operating manual.
- `AGENTS.md` mirroring `CLAUDE.md` for non-Claude agents.

Cascade is verified by:

```
./scripts/verify-governance-cascade.sh
```

Last verified at 39/39 governance files (Phase 4 close-out, commit 21e6686).

11. Build & Test

Command	Purpose
<code>make build</code>	Inner module: produce <code>bin/helixcode</code> and <code>bin/cli</code>

Command	Purpose
<code>make verify-compile</code>	Compile-only smoke (helix_code/Makefile)
<code>make test</code>	All unit tests
<code>make test-coverage</code>	Coverage with <code>-race</code>
<code>make lint</code>	golangci-lint + CONST-046 string lint
<code>make test-infra-up</code>	Start real PostgreSQL/Redis/Ollama stack
<code>make test-full</code>	All tests, zero skips
<code>make demo-all</code>	Run every submodule's demo
<code>./scripts/no-silent-skips.sh</code>	Fail on bare <code>t.Skip()</code> without ticket

12. Commit & Push

- **Always SSH URLs.** No HTTPS for git (Constitution Rule 3).
- **Push to four remotes:** origin, github, gitlab, upstream (CONST-043: no force pushes; no history rewrites).
- **Commit messages** follow the format used in recent commits — see `git log --oneline -10`.
- **Co-Authored-By trailer** for AI-generated commits.

13. Reference Material

- Architecture: docs/improvements/06_diagrams_real/ (verified diagrams)
- ADRs: docs/adr/
- Bluff-proofing history: docs/bluff_proofing/
- LLMs verifier integration: docs/llms_verifier/
- helix_qa integration: docs/helix_qa/
- Phase-by-phase evidence: docs/improvements/PROGRESS.md

Sources verified

Sources verified 2026-05-29: <https://go.dev/doc/devel/release> (Go 1.26.0 GA 2026-02-10, latest patch go1.26.3 2026-05-07 — confirms CLAUDE.md §3.1 inner-module Go 1.26), <https://endoflife.date/postgresql> (PostgreSQL 15 supported through 2027-11 — confirms the “PostgreSQL with Redis caching” stack), <https://endoflife.date/redis> (Redis 7.4 still receives maintenance/security patches — confirms the real-Redis test stack). Confirmed: golangci-lint, `make test-infra-up` real-stack guidance, and four-remote push (CONST-043) match project authority (CLAUDE.md §3.1/§3.4). Negative finding: no public external service endpoints are documented in this developer-guide README — the only URL is the operator-local OTEL collector `http://localhost:4317` (illustrative localhost, not a public endpoint, nothing to cross-reference).